

Manipulación del DOM en JavaScript

El **Document Object Model (DOM)** es la interfaz de programación que representa un documento HTML como una estructura en árbol de nodos dentro de la memoria del navegador. Permite a JavaScript leer, modificar, agregar o eliminar elementos, atributos y estilos del documento de manera dinámica e interactiva.

Índice

- [1. El Árbol del DOM y tipos de nodos](#)
 - [2. Selección de elementos](#)
 - [3. Manipulación de contenido y atributos](#)
 - [4. Manipulación de estilos y clases CSS](#)
 - [5. Creación, inserción y eliminación de nodos](#)
 - [6. Gestión de eventos](#)
 - [7. Cuadro comparativo de operaciones frecuentes](#)
-

1. El árbol del DOM y tipos de nodos

Cuando el navegador carga una página web, interpreta el código HTML y lo transforma en una jerarquía de objetos en memoria. En este modelo, cada componente del documento es un nodo dentro del árbol:

- **Document:** el nodo raíz que representa la totalidad del documento HTML cargado (document).
 - **Element:** nodos que corresponden a cada etiqueta HTML (<div>, <button>, <h1>, etc.).
 - **Text:** nodos que contienen el texto dentro de un elemento HTML.
 - **Attribute:** nodos que representan las propiedades de una etiqueta HTML (como src, href, class o id).
-

2. Selección de elementos

Antes de realizar modificaciones en la interfaz, JavaScript necesita ubicar los nodos deseados dentro del árbol. El estándar moderno para seleccionar elementos utiliza la sintaxis de selectores CSS.

Métodos principales de selección

- **document.querySelector(selector):** busca y devuelve el **primer elemento** que coincida con el selector CSS especificado. Si no encuentra coincidencias, retorna

null.

- **document.querySelectorAll(selector):** busca y devuelve una lista con todos los elementos que coinciden con el selector en formato de colección (NodeList).

```
// Selección por ID
const titulo = document.querySelector("#titulo-principal");

// Selección por clase (devuelve todos los elementos coincidentes)
const botones = document.querySelectorAll(".btn-accion");

// Selección por etiqueta
const primerParrafo = document.querySelector("p");
```

3. Manipulación de contenido y atributos

Una vez seleccionado un nodo del DOM, se puede modificar su contenido textual, su marcado HTML o sus atributos.

Propiedades de contenido

- **textContent:** permite obtener o modificar únicamente el texto plano de un elemento, sin interpretar etiquetas HTML. Es el método recomendado para manejar texto de forma segura.
- **innerHTML:** permite obtener o reemplazar el marcado HTML interno de un elemento. Se debe usar con precaución para prevenir vulnerabilidades de seguridad (*Cross-Site Scripting* o XSS).

```
const contenedor = document.querySelector("#mensaje");

// Asignación de texto plano seguro
contenedor.textContent = "La operación fue completada con éxito.";

// Modificación incluyendo marcado HTML
contenedor.innerHTML = "<strong>Atención:</strong> Los cambios no guardados se perderán.";
```

Manipulación de atributos

Los atributos de los elementos HTML se pueden inspeccionar, agregar o remover mediante métodos dedicados:

- **getAttribute(nombre):** obtiene el valor actual de un atributo.
- **setAttribute(nombre, valor):** asigna o modifica un atributo en el elemento.
- **removeAttribute(nombre):** elimina un atributo del elemento.

```
const enlace = document.querySelector("a");

// Configuración de atributos HTML
```

```
enlace.setAttribute("href", "https://ejemplo.com");
enlace.setAttribute("target", "_blank");
```

4. Manipulación de estilos y clases CSS

Aunque es posible modificar estilos directamente desde JavaScript, la buena práctica establece que las reglas visuales deben ser controladas alternando clases CSS.

Gestión de clases (classList)

La propiedad `classList` provee una API simple para gestionar las clases CSS asignadas a un nodo:

- **`classList.add('clase')`**: agrega una clase al elemento.
- **`classList.remove('clase')`**: elimina una clase del elemento.
- **`classList.toggle('clase')`**: alterna la presencia de una clase (la remueve si ya la tiene, o la agrega si no está).
- **`classList.contains('clase')`**: verifica si el elemento posee una clase en particular y retorna un booleano (true/false).

```
const tarjeta = document.querySelector(".card");

// Alteración dinámica de estilos mediante clases
tarjeta.classList.add("destacada");
tarjeta.classList.toggle("oculta");
```

5. Creación, inserción y eliminación de nodos

El árbol del DOM se puede modificar dinámicamente agregando o removiendo nodos durante la ejecución de la página.

Creación e inserción de elementos

Para agregar un nuevo elemento visual a la página se debe seguir un proceso de tres pasos:

```
// 1. Crear el elemento en memoria
const nuevoItem = document.createElement("li");

// 2. Configurar sus propiedades y contenido
nuevoItem.textContent = "Nueva tarea pendiente";
nuevoItem.classList.add("item-lista");

// 3. Seleccionar el elemento padre e insertar el nuevo nodo hijo
const lista = document.querySelector("#lista-tareas");
lista.appendChild(nuevoItem);
```

Eliminación de nodos

Para borrar un nodo del árbol del DOM, se utiliza el método `remove()` directamente sobre el elemento seleccionado.

```
const elementoABorrar = document.querySelector("#bloque-  
publicidad");  
  
// Eliminación directa del nodo  
elementoABorrar.remove();
```

6. Gestión de eventos

Los eventos son las acciones o interacciones que ocurren en la página web (como clics del usuario, tipeo en el teclado o el envío de un formulario). JavaScript puede registrar escuchas (*listeners*) para ejecutar funciones de respuesta en consecuencia.

Escuchar eventos (addEventListener)

El método `addEventListener` registra una función que se ejecutará en respuesta a un evento determinado.

```
const botonGuardar = document.querySelector("#btn-guardar");  
  
// Registro del evento 'click'  
botonGuardar.addEventListener("click", (event) => {  
  console.log("El usuario hizo clic en el botón.");  
  console.log("Elemento objetivo que detonó el evento:",  
    event.target);  
});
```

Manejo de formularios y preventDefault

Al trabajar con formularios, el comportamiento por defecto del navegador al enviar los datos es recargar la página. Se utiliza `event.preventDefault()` para interceptar este comportamiento y manejar el procesamiento del formulario mediante código asíncrono en JavaScript.

```
const formulario = document.querySelector("#form-registro");  
  
formulario.addEventListener("submit", (event) => {  
  // Evita el envío tradicional por HTTP y la recarga de la  
  // página  
  event.preventDefault();  
  
  const inputUsuario = document.querySelector("#input-  
usuario").value;  
  console.log("Procesando formulario para el usuario:",  
    inputUsuario);  
});
```

7. Cuadro Comparativo de Operaciones Frecuentes

Categoría	Operación	Método / Propiedad	Ejemplo de uso
Búsqueda	Seleccionar un nodo	querySelector()	document.querySelector('.tarjeta')
Búsqueda	Seleccionar múltiples nodos	querySelectorAll()	document.querySelectorAll('div')
Contenido	Modificar texto plano	textContent	elem.textContent = 'Guardado'
Atributos	Asignar atributo HTML	setAttribute()	elem.setAttribute('disabled', true)
Estilos	Alternar clase CSS	classList.toggle()	elem.classList.toggle('activo')
Estructura	Crear nuevo nodo	createElement()	document.createElement('p')
Estructura	Insertar nodo hijo	appendChild()	padre.appendChild(hijo)
Estructura	Eliminar nodo	remove()	elem.remove()
Interacción	Escuchar un evento	addEventListener()	elem.addEventListener('click', fn)